

E-Commerce Order Analysis - Step-by-Step Code Explanation

This class analyzes e-commerce orders using **Pandas**. The best way to understand it is to see how the methods would be developed one by one.

1. Create the Class

We first create a class to hold all order-related analysis methods.

```
import pandas as pd
```

```
class EcommerceAnalyzer:  
    pass
```

At this stage, the class does nothing.

2. Create DataFrame Method

Goal

Convert raw order data into a Pandas DataFrame.

Input

```
data = [  
    [101, "Electronics", "2025-01-10", "Delivered"],  
    [101, "Books", "2025-01-15", "Returned"]  
]
```

Method

```
def create_order_df(self, data: list) -> pd.DataFrame:  
    return pd.DataFrame(  
        data, columns=["id", "category", "date", "status"]  
    )
```

```
data,
columns=[
    "CustomerID",
    "Category",
    "OrderDate",
    "OrderStatus"
]
)
```

Execution

```
analyzer = EcommerceAnalyzer()
```

```
df = analyzer.create_order_df(data)
```

```
print(df)
```

Output

	CustomerID	Category	OrderDate	OrderStatus
0	101	Electronics	2025-01-10	Delivered
1	101	Books	2025-01-15	Returned

3. Monthly Delivery Rate

Business Requirement

Find:

Delivery Rate =
 $(\text{Delivered Orders} / \text{Total Orders}) \times 100$

For every:

- Customer
 - Month
-

Step 3.1 Copy Original Data

Never modify the original DataFrame.

```
temp = df.copy()
```

Example

Original DF

CustomerID	OrderDate	OrderStatus
101	2025-01-10	Delivered
101	2025-01-15	Returned

Step 3.2 Extract Month

Take only Year-Month.

```
temp["Month"] = temp["OrderDate"].str[:7]
```

Example

2025-01-10 → 2025-01

2025-01-15 → 2025-01

Result

CustomerID	OrderDate	Month
101	2025-01-10	2025-01
101	2025-01-15	2025-01

Step 3.3 Count Total Orders

```
total_orders = (  
    temp.groupby(  
        ["CustomerID", "Month"]  
    )  
    .size()  
    .reset_index(name="TotalOrders")  
)
```

)

GroupBy

```
temp.groupby(["CustomerID","Month"])
```

Groups:

Customer 101, Month 2025-01

Count Rows

```
.size()
```

Output

CustomerID	Month	TotalOrders
101	2025-01	2

Step 3.4 Count Delivered Orders

Filter only delivered orders.

```
temp[temp["OrderStatus"] == "Delivered"]
```

Result

CustomerID	Month
101	2025-01

Count them:

```
delivered_orders = (  
    temp[temp["OrderStatus"] == "Delivered"]  
    .groupby(  
        ["CustomerID", "Month"]  
    )  
    .size()  
    .reset_index(name="DeliveredOrders")  
)
```

Output

CustomerID	Month	DeliveredOrders
101	2025-01	1

Step 3.5 Merge Results

Combine TotalOrders and DeliveredOrders.

```
result = total_orders.merge(
    delivered_orders,
    on=["CustomerID", "Month"],
    how="left"
)
```

Output

CustomerID	Month	TotalOrders	DeliveredOrders
101	2025-01	2	1

Step 3.6 Handle Missing Values

Suppose a customer has no delivered orders.

NaN

becomes

```
0
result["DeliveredOrders"] = (
    result["DeliveredOrders"]
    .fillna(0)
    .astype(int)
)
```

Step 3.7 Calculate Delivery Rate

Formula:

```
result["Delivery Rate"] = (  
    result["DeliveredOrders"]  
    /  
    result["TotalOrders"]  
    *  
    100  
)
```

Example

1 Delivered
2 Total

Calculation

$1/2 * 100 = 50\%$

Output

CustomerID	Month	Delivery Rate
101	2025-01	50

Final Method

```
def monthly_delivery_rate(  
    self,  
    df: pd.DataFrame  
) -> pd.DataFrame:  
  
    temp = df.copy()  
  
    temp["Month"] = temp["OrderDate"].str[:7]  
  
    total_orders = (  
        temp.groupby(["CustomerID", "Month"])  
        .size()  
        .reset_index(name="TotalOrders")  
    )  
  
    delivered_orders = (  
        temp.groupby(["CustomerID", "Month"])  
        .size()  
        .reset_index(name="DeliveredOrders")  
    )  
  
    return total_orders, delivered_orders
```

```

temp[temp["OrderStatus"] == "Delivered"]
.groupby(["CustomerID", "Month"])
.size()
.reset_index(name="DeliveredOrders")
)

result = total_orders.merge(
    delivered_orders,
    on=["CustomerID", "Month"],
    how="left"
)

result["DeliveredOrders"] = (
    result["DeliveredOrders"]
    .fillna(0)
    .astype(int)
)

result["Delivery Rate"] = (
    result["DeliveredOrders"]
    /
    result["TotalOrders"]
    *
    100
)

return result[
    ["CustomerID", "Month", "Delivery Rate"]
]

```

4. Add Return Flag

Business Requirement

Create a binary column:

Returned → 1

Others → 0

Step 4.1 Copy Data

```
result = df.copy()
```

Step 4.2 Create Boolean Condition

```
result["OrderStatus"] == "Returned"
```

Output

```
True  
False  
True
```

Step 4.3 Convert Boolean to Integer

```
.astype(int)
```

Conversion

```
True → 1  
False → 0
```

Final Method

```
def add_return_flag(  
    self,  
    df: pd.DataFrame  
) -> pd.DataFrame:  
  
    result = df.copy()  
  
    result["IsReturned"] = (  
        result["OrderStatus"] == "Returned"  
    ).astype(int)  
  
    return result
```

Output

OrderStatus	IsReturned
Delivered	0
Returned	1

Cancelled 0

5. Frequent Returners

Business Requirement

Find customers who return products frequently.

Step 5.1 Filter Returned Orders

```
df[df["OrderStatus"] == "Returned"]
```

Output

CustomerID

101

101

102

Step 5.2 Count Returns

```
.groupby("CustomerID")  
.size()
```

Output

CustomerID ReturnCount

101 2

102 1

Step 5.3 Convert to DataFrame

```
.reset_index(name="ReturnCount")
```

Step 5.4 Apply Threshold

Suppose:

threshold = 1

Then:

```
result[result["ReturnCount"] > threshold]
```

Output

CustomerID	ReturnCount
101	2

Final Method

```
def frequent_returners(
    self,
    df: pd.DataFrame,
    threshold: int
) -> pd.DataFrame:

    result = (
        df[df["OrderStatus"] == "Returned"]
        .groupby("CustomerID")
        .size()
        .reset_index(name="ReturnCount")
    )

    return result[
        result["ReturnCount"] > threshold
    ]
```

6. Category Order Summary

Business Requirement

Show status counts per category.

Example:

Category	Delivered	Cancelled	Returned
Books	10	2	1
Electronics	20	5	3

Step 6.1 Use Crosstab

```
pd.crosstab(  
    df["Category"],  
    df["OrderStatus"]  
)
```

Input

Category	Status
Books	Delivered
Books	Returned
Books	Delivered

Output

Category	Delivered	Returned
Books	2	1

Step 6.2 Ensure Fixed Column Order

```
summary = summary.reindex(  
    columns=[  
        "Delivered",  
        "Cancelled",  
        "Returned"  
    ],  
    fill_value=0  
)
```

Why?

Some datasets may not contain all statuses.

Without reindex:

Delivered Returned

With reindex:

Delivered Cancelled Returned

Always consistent.

Step 6.3 Convert Index to Column

```
summary.reset_index()
```

Output

Category	Delivered	Cancelled	Returned
Books	2	0	1

Final Method

```
def category_order_summary(  
    self,  
    df: pd.DataFrame
```

) -> pd.DataFrame:

```
summary = pd.crosstab(  
    df["Category"],  
    df["OrderStatus"]  
)  
  
summary = summary.reindex(  
    columns=[  
        "Delivered",  
        "Cancelled",  
        "Returned"  
    ],  
    fill_value=0  
)  
  
return summary.reset_index()
```

Complete Development Sequence

When building this class from scratch, the logical order is:

1. Create EcommerceAnalyzer class

2. create_order_df()

↓

Convert raw data into DataFrame

3. add_return_flag()

↓

Add derived column

4. frequent_returners()

↓

Analyze customer returns

5. category_order_summary()

↓

Analyze category performance

6. monthly_delivery_rate()

↓

Advanced KPI calculation using GroupBy + Merge + Aggregation

This sequence follows a common data engineering workflow:

Raw Data



DataFrame Creation



Feature Engineering



Customer Analytics



Category Analytics



Business KPI Analytics